
DATA WAREHOUSING ASSIGNMENT

High-Level Data Warehouse Design

Movie Rental System

Field	Detail
Course	Data Warehousing
Name	Rama Sabboubbeh
Topic	From OLTP Schema to Dimensional Model and ETL Design
Source OLTP	Sakila MySQL Sample Database
DW Database	sakila_dw (Star Schema)

Section 1 — Introduction

This assignment addresses the design and implementation of a Data Warehouse (DW) for movie rental business. The OLTP database is optimized for daily transactional operations - recording rentals, processing payments, managing customers and inventory. While it excels at these tasks, its normalized structure makes analytical reporting slow and complex.

The goal of this project is to redesign the data model into a dimensional (analytical) structure using star schema principles, build an ETL pipeline to populate it, and use the populated warehouse to answer specific business questions.

1.1 OLTP vs Data Warehouse

Aspect	OLTP (sakila)	Data Warehouse (sakila_dw)
Purpose	Daily transactions	Analytical reporting
Schema	Highly normalized (3NF)	Denormalized (star schema)
Optimization	Fast INSERT/UPDATE	Fast SELECT/GROUP BY
Tables	15+ related tables	3 fact + 7 dimension tables
Users	Clerks, applications	Managers, analysts
Data volume	Current state only	Historical + current
Query type	Row-level lookups	Aggregations, trends

Section 2 — Business Questions

The following 15 analytical questions drive the design of the data warehouse. Each question targets a specific business domain.

#	Business Question	Domain
Q1	Which films are rented most frequently?	Film Popularity
Q2	Which films generate the highest revenue?	Revenue
Q3	Which film categories are most popular?	Film Popularity
Q4	Which stores generate the highest number of rentals?	Store Performance
Q5	Which stores generate the highest revenue?	Store Performance
Q6	Which customers rent the most films?	Customer Behavior
Q7	Which customers generate the highest revenue?	Customer Behavior
Q8	How does rental activity change over time?	Trends
Q9	How does revenue change by month, quarter, or year?	Trends
Q10	Which staff members process the most rentals and payments?	Staff Performance
Q11	Which cities or countries generate the highest customer activity?	Location
Q12	What is the average rental duration for films or categories?	Operations
Q13	Which films are returned late most often?	Operations
Q14	How does store performance differ by location?	Store Performance
Q15	What are the most active customer locations?	Location

Section 3 — Dimensional Model Design

3.1 Selected Business Processes

Three core business processes were identified from the OLTP schema and converted into fact tables:

1. Film Rental Transactions - a customer rents a film from a store (rental table)
2. Payment Transactions - a customer makes a payment for a rental (payment table)
3. Inventory Snapshots - daily availability of film copies per store (inventory + rental tables)

3.2 Fact Tables

fact_rental — Film Rental Transactions

Grain: One row per rental transaction.

Column	Type	Description
rental_key	INT PK	Surrogate key (auto-increment)
rental_date_key	INT FK	FK → dim_date (rental date)
return_date_key	INT FK	FK → dim_date (return date, nullable)
customer_key	INT FK	FK → dim_customer
film_key	INT FK	FK → dim_film
category_key	INT FK	FK → dim_category
store_key	INT FK	FK → dim_store
staff_key	INT FK	FK → dim_staff
rental_id	INT	Natural key from OLTP
rental_duration_days	SMALLINT	Actual days kept
allowed_duration_days	TINYINT	Days allowed by rental plan
days_overdue	SMALLINT	Days past allowed (0 if on time, NULL if not returned)
rental_rate	DECIMAL	Rate charged per rental
replacement_cost	DECIMAL	Cost to replace the film
is_returned	TINYINT	1 = returned, 0 = still out

fact_payment — Payment Transactions

Grain: One row per payment transaction.

Column	Type	Description
payment_key	INT PK	Surrogate key
payment_date_key	INT FK	FK → dim_date
customer_key	INT FK	FK → dim_customer

Column	Type	Description
staff_key	INT FK	FK → dim_staff
store_key	INT FK	FK → dim_store
rental_key	INT FK	FK → fact_rental (optional link)
payment_id	SMALLINT	Natural key from OLTP
amount_paid	DECIMAL	Payment amount (measure)

fact_inventory_snapshot — Daily Inventory State

Grain: One row per film × store × active rental date.

Column	Type	Description
snapshot_key	INT PK	Surrogate key
snapshot_date_key	INT FK	FK → dim_date
film_key	INT FK	FK → dim_film
store_key	INT FK	FK → dim_store
total_copies	SMALLINT	Total copies owned
copies_rented_out	SMALLINT	Currently on loan
copies_available	SMALLINT	Currently in stock

3.3 Dimension Tables

Dimension	Main Attributes	Source OLTP Tables	Purpose
dim_date	date_key, year, quarter, month, day_name, is_weekend	Generated (no OLTP source)	Enables all time-based analysis
dim_customer	customer_id, name, email, address, city, country, active (SCD2)	customer, address, city, country	Customer behavior and location analysis
dim_film	film_id, title, language, rating, rental_rate, length	film, language	Film performance and characteristics
dim_category	category_id, name	category	Category-level popularity analysis
dim_store	store_id, address, city, country, manager_name	store, staff, address, city, country	Store location and performance
dim_staff	staff_id, full_name, username, store_id	staff	Staff productivity analysis
dim_actor	actor_id, full_name	actor	Actor-film association (via bridge)
bridge_film_actor	film_key, actor_key	film_actor	Resolves film-actor many-to-many

3.4 Shared Dimensions

Dimension	Shared By
dim_date	fact_rental, fact_payment, fact_inventory_snapshot
dim_customer	fact_rental, fact_payment
dim_film	fact_rental, fact_inventory_snapshot
dim_store	fact_rental, fact_payment, fact_inventory_snapshot
dim_staff	fact_rental, fact_payment

3.5 Schema Choice: Star Schema

A star schema was selected because:

- Queries are simpler - analysts write fewer JOINS
 - BI tools optimize best against star schemas
 - Location data (city, country) is folded into dim_customer and dim_store - no separate dim_location snowflake needed
 - Language is folded into dim_film - no separate dim_language needed
 - The one exception is bridge_film_actor which handles the film-actor many-to-many relationship
-

Section 4 — Dimensional Model Diagram

The star schema below shows three fact tables at the center, surrounded by seven dimension tables and one bridge table. Solid lines indicate primary fact-dimension relationships; shared dimensions connect to multiple facts.

Star Schema Summary:

Table	Type	Primary Key	Foreign Keys
fact_rental	FACT	rental_key	rental_date_key, return_date_key, customer_key, film_key, category_key, store_key, staff_key
fact_payment	FACT	payment_key	payment_date_key, customer_key, staff_key, store_key, rental_key
fact_inventory_snapshot	FACT	snapshot_key	snapshot_date_key, film_key, store_key
dim_date	DIMENSION	date_key	—
dim_customer	DIMENSION	customer_key	—
dim_film	DIMENSION	film_key	—
dim_category	DIMENSION	category_key	—
dim_store	DIMENSION	store_key	—
dim_staff	DIMENSION	staff_key	—
dim_actor	DIMENSION	actor_key	—
bridge_film_actor	BRIDGE	(film_key, actor_key)	film_key → dim_film, actor_key → dim_actor

Note: The full visual diagram in “diagram.mwb”

Section 5 — ETL Design

The ETL process is implemented in Python using pandas and SQLAlchemy (02_sakila_etl.ipynb). It connects to both the “sakila” OLTP database and the “sakila_dw” warehouse.

5.1 Extract

The following 15 OLTP tables are extracted using `pd.read_sql()`:

OLTP Table	Why Extracted
customer	Core customer data - name, email, active status
address	Customer and store address details
city	City names linked to addresses
country	Country names linked to cities
film	Film attributes - title, rate, duration, rating
language	Film language names
category	Film genre categories
film_category	Maps films to categories (many-to-many)
store	Store locations and manager references
staff	Staff names, usernames, store assignments
actor	Actor names
film_actor	Maps films to actors (many-to-many)
inventory	Tracks physical film copies per store
rental	Core rental transactions
payment	Payment transactions linked to rentals

5.2 Transform

Key transformations applied before loading:

1. Table Joining — customer + address + city + country merged into one flat `dim_customer` row. Store similarly flattened.
 2. Column Selection — only columns needed in the DW are kept after merging to avoid unknown column errors on load.
 3. Full Name Computation — `CONCAT(first_name, ' ', last_name)` computed for staff and actor dimensions.
 4. NULL Handling — missing emails filled with 'no-email@unknown.com', missing postal codes with '00000'.
 5. Date Key Generation — `rental_date`, `return_date`, `payment_date` converted to YYYYMMDD integer keys.
 6. Type Fixing — `original_language_id` cast to nullable `Int64` to handle NULL values before merging.
 7. Duplicate Removal — `drop_duplicates()` applied on every dimension's natural key.
-

-
8. Measures Calculated — rental_duration_days, days_overdue, is_returned computed during transform.
 9. Surrogate Key Resolution — after dimensions are loaded, their auto-increment keys are read back and joined to fact DataFrames before loading.
 10. Bridge Table — film_actor OLTP rows mapped to DW surrogate keys via dim_film and dim_actor lookups.

5.3 Load

Loading order — dimensions first, then bridge, then facts:

Step	Table Loaded	Reason
1	dim_date	No FK dependencies — generated calendar
2	dim_customer	No FK dependencies
3	dim_film	No FK dependencies
4	dim_category	No FK dependencies
5	dim_store	No FK dependencies
6	dim_staff	No FK dependencies
7	dim_actor	No FK dependencies
8	bridge_film_actor	Requires dim_film and dim_actor surrogate keys
9	fact_rental	Requires all dimensions loaded (FK constraints)
10	fact_payment	Requires dim_* and fact_rental surrogate keys
11	fact_inventory_snaps hot	Requires dim_date, dim_film, dim_store

Surrogate Key Strategy: Dimensions use AUTO_INCREMENT primary keys. After each dimension is loaded, its keys are read back with SELECT dim_key, natural_key FROM dim_table and joined to fact DataFrames before fact tables are inserted.

Re-run Safety: All tables are TRUNCATED (in reverse FK order) before each load run. This prevents duplicate primary key errors and ensures a clean reload.

Data Quality Checks: ASSERT statements verify row counts after loading (e.g., assert len(dim_customer)==599). Any mismatch raises an error before proceeding to facts.

Section 6 — Data Quality Considerations

Rule	Check	Action on Failure
No NULL natural keys	customer_id, film_id, rental_id must not be NULL	Drop row, log warning
No duplicate natural keys	Each customer_id appears once per is_current=1	Deduplicate before load
Valid date keys	rental_date_key must exist in dim_date	Log error, skip row
Return after rental	return_date >= rental_date	Set return_date to NULL (unreturned)
Positive measures	amount_paid > 0, rental_rate > 0	Flag and log invalid rows
FK referential integrity	All fact FKs must exist in their dimension	MySQL FK constraints enforce this
Row count validation	dim_customer=599, dim_film=1000, fact_rental~16044	Assert after load, abort if mismatch
Non-negative overdue	days_overdue >= 0 (not negative)	CASE WHEN to floor at 0

Section 7 — Conclusion

This assignment successfully transformed the Sakila OLTP schema into a fully functional analytical data warehouse using dimensional modeling principles.

Key achievements:

- Designed a star schema with 3 fact tables, 7 dimension tables, and 1 bridge table
- Built a complete Python ETL pipeline (pandas + SQLAlchemy) that extracts from sakila, transforms data, and loads sakila_dw
- Answered all 15 business questions using SQL queries against the populated fact tables
- Each business question is supported by both a result table and a chart visualization
- Applied SCD Type 2 on dim_customer for historical change tracking
- Implemented data quality checks and re-run safety (TRUNCATE before reload)

Business Value:

The populated data warehouse enables business managers to answer questions in seconds that would require complex multi-table joins in the OLTP system. Film popularity, revenue trends, store performance, customer behavior, and staff productivity can all be analyzed with simple GROUP BY queries against the fact tables. The star schema structure is compatible with all major BI tools (Tableau, Power BI, Excel pivot tables).

Deliverable	File	Description
Star Schema DDL	01_sakila_dw_structure.sql	Creates sakila_dw database with all tables and indexes
ETL Pipeline	02_sakila_etl.ipynb	Python notebook: Extract → Transform → Load
Business Analysis	03_business_analysis.ipynb	15 business questions with tables and charts
Assignment Report	Assignment_Report.docx	This document
README	README.md	GitHub documentation
diagram	diagram.mwb	Diagram for sakila_dw database